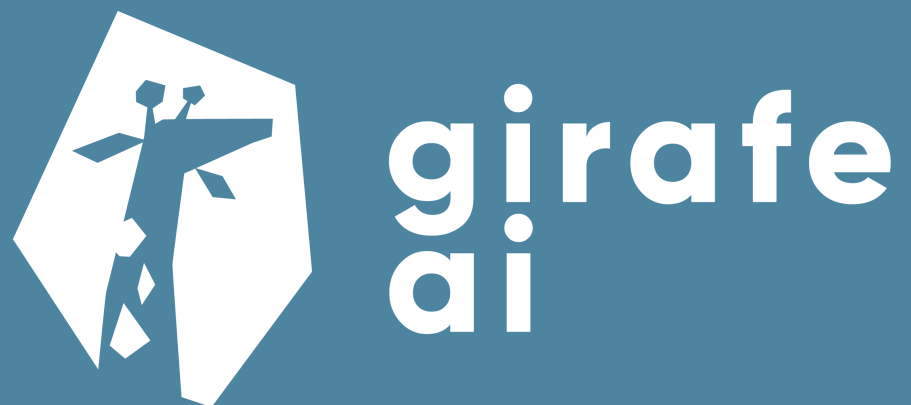


Model deploy Nvidia Triton Server

Vladislav Goncharenko

MIPT, fall 2025



Inference servers overview

girafe
ai

01

Requirements for inference server

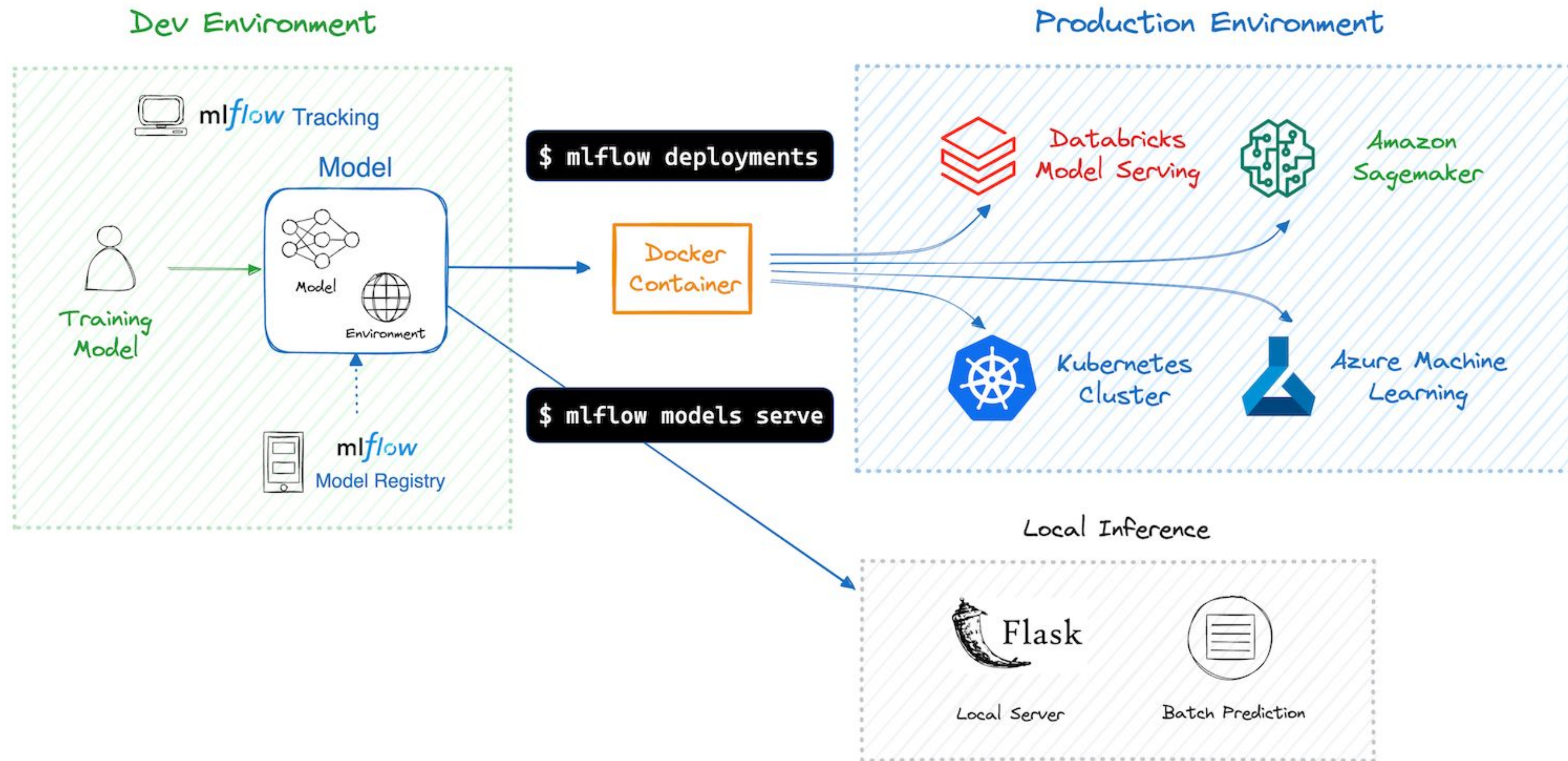
- common API
 - REST, gRPC, websockets, ...
- request balancer (queue)
- monitoring
 - Grafana, Kibana, etc
- ease of preparation for ML engineers

Possible solutions

- MLflow
 - <https://mlflow.org/docs/latest/deployment/>
 - https://mlflow.org/docs/latest/api_reference/python_api/mlflow.pyfunc.html
- Triton
- Seldon
- ClearML
- etc...



MLflow Serving



Triton inference server

girafe
ai

02

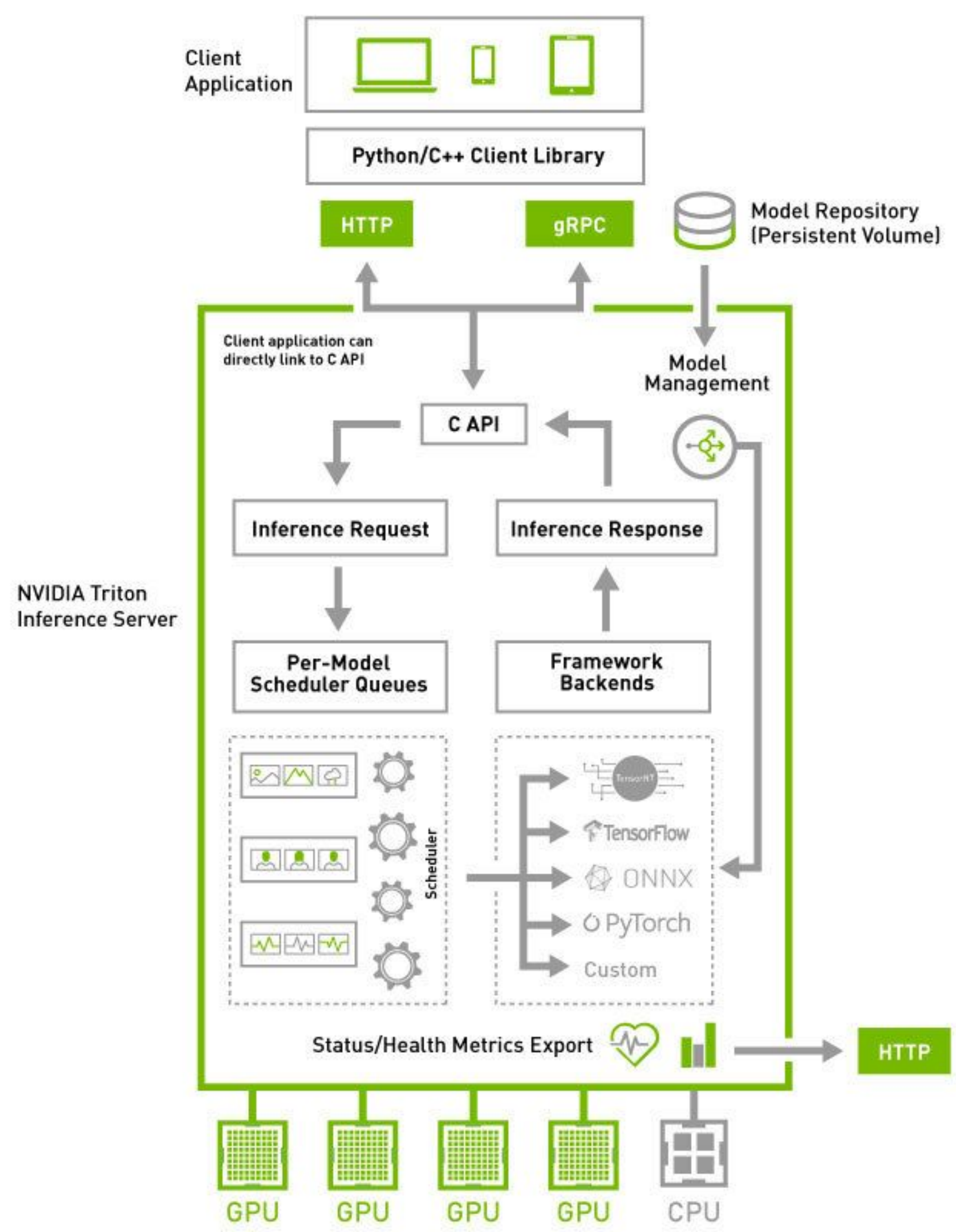
Why Triton?

- Open source
 - BSD license
- Developed by Nvidia (good maintenance)
- C++
- Enterprise grade, security & api stability
- Concurrent model execution
- Dynamic batching
- Many supported frameworks out of the box
- Live updates
- Allowed both CPU and GPU instances
- Allowed multiple instances for the same model
- Support for arbitrary function execution
- Model ensembles out of the box
- Optimized and certified to deploy anywhere
 - Cloud, Datacenter, Edge,...

Triton Inference Server

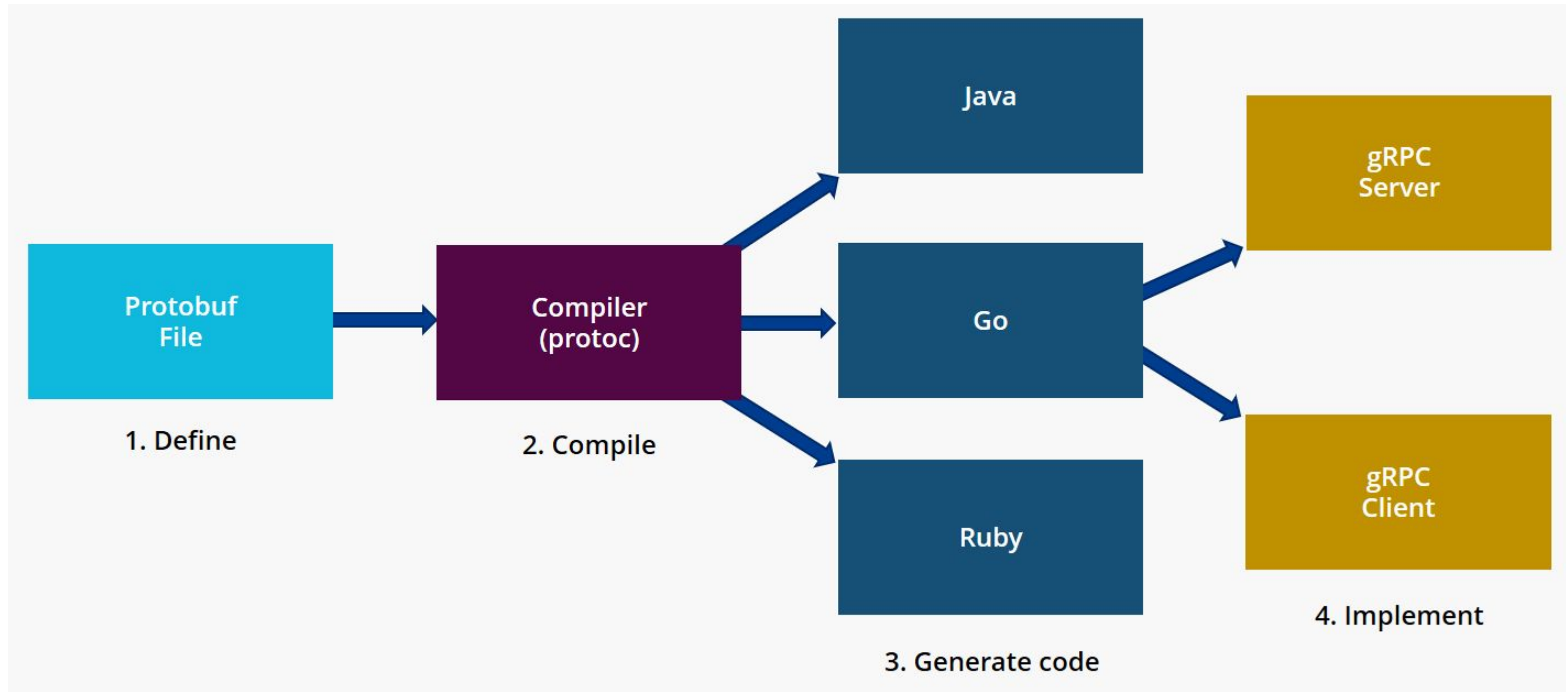
- documentation
https://github.com/triton-inference-server/server/tree/main/docs/user_guide
- code <https://github.com/triton-inference-server/server>
- 7.4k stars on github
- written in C++ and Python
- used by many companies for NNs inference

Architecture

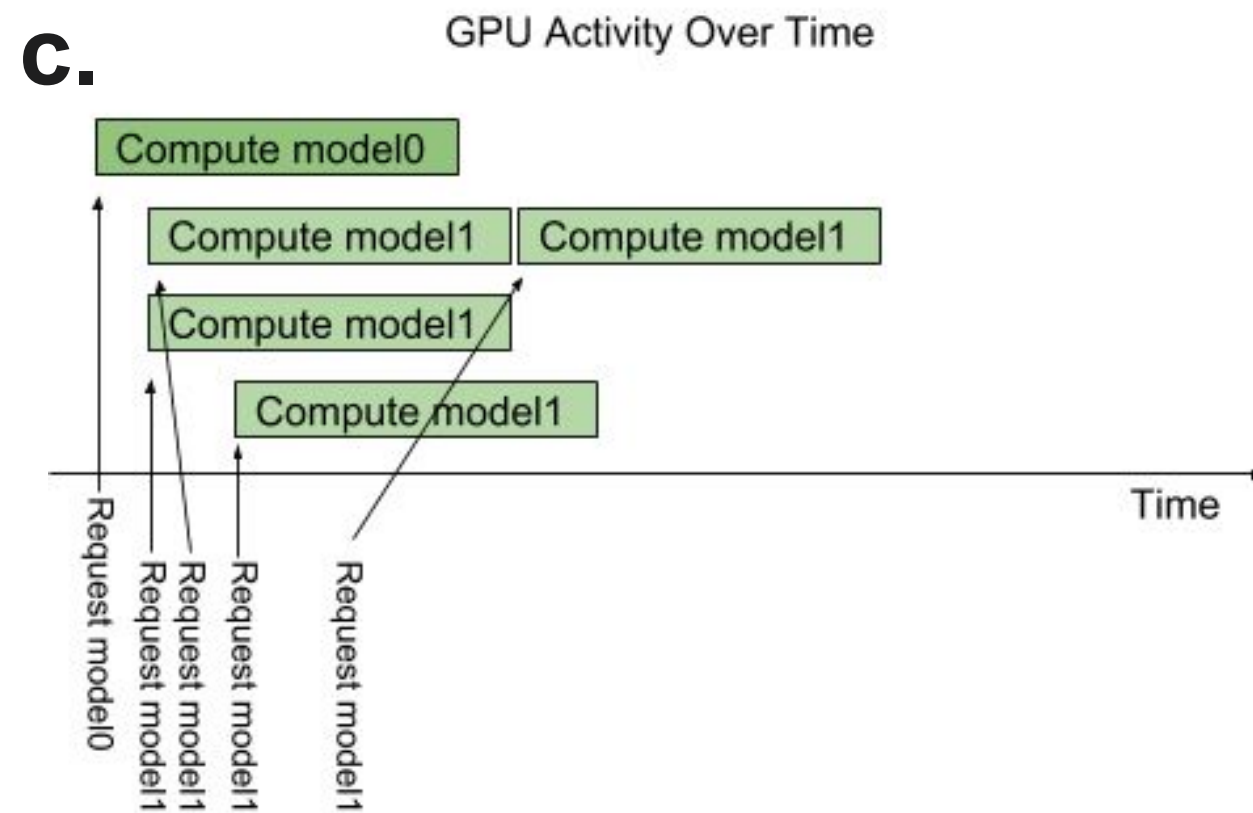
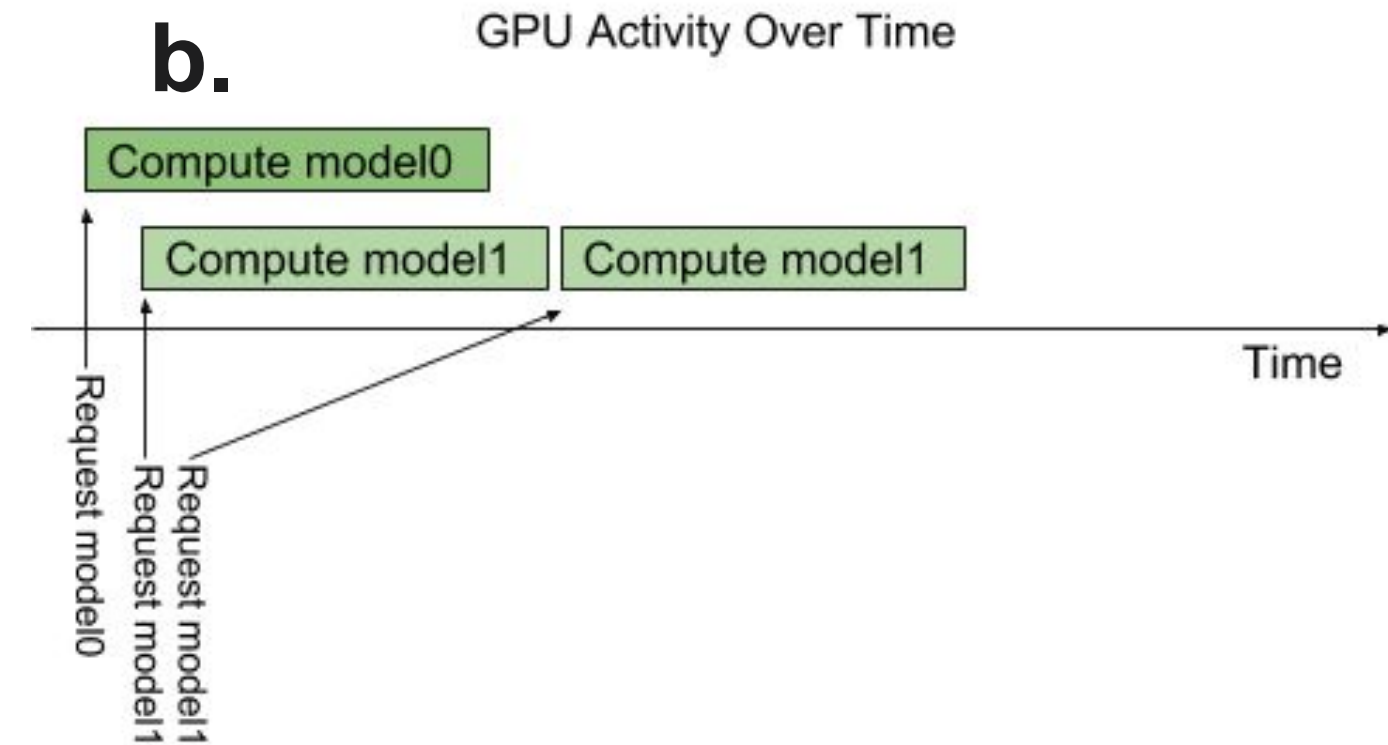
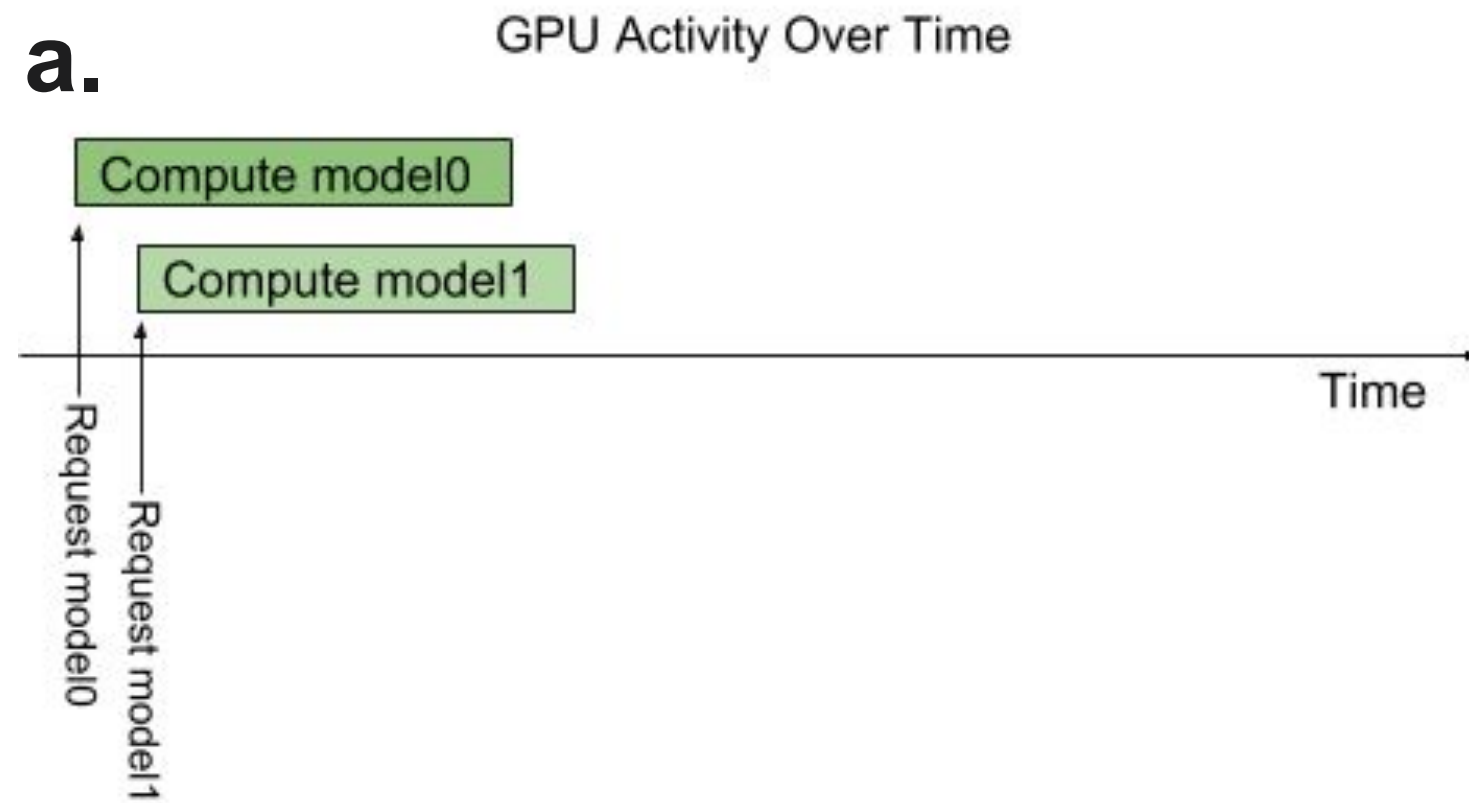


gRPC

- <https://grpc.io/>
- https://en.wikipedia.org/wiki/Remote_procedure_call
- RPC explained https://www.youtube.com/watch?v=K4_cgtAe4HM



Concurrent Execution



Model repository

```
<model-repository-path>/  
  <model-name>/  
    [config.pbtxt]  
    [<output-labels-file> ...]  
  <version>/  
    <model-definition-file>  
  <version>/  
    <model-definition-file>  
  ...  
<model-name>/  
  [config.pbtxt]  
  [<output-labels-file> ...]  
  <version>/  
    <model-definition-file>  
  <version>/  
    <model-definition-file>  
  ...  
...
```

- https://github.com/girafe-ai/mlops/tree/master/09_nvidia_triton/01_huggingface_transformer
- https://github.com/girafe-ai/mlops/tree/25s_mipt/triton

Model configuration

```
1  platform: "tensorrt_plan"
2  max_batch_size: 8
3
4
5  input [
6      {
7          name: "input0"
8          data_type: TYPE_FP32
9          dims: [ 16 ]
10     },
11     {
12         name: "input1"
13         data_type: TYPE_FP32
14         dims: [ 16 ]
15     }
16 ]
17
18 output [
19     {
20         name: "output0"
21         data_type: TYPE_FP32
22         dims: [ 16 ]
23     }
24 ]
```

Explicit/implicit batch size

```
1 platform: "tensorrt_plan"
2 max_batch_size: 8
```

```
5 input [
6     {
7         name: "input"
8         data_type: TYPE_FP32
9         dims: [ 16 ]
10    }
11]
```

```
13 output [
14     {
15         name: "output"
16         data_type: TYPE_FP32
17         dims: [ 16 ]
18    }
19]
```

```
1 platform: "tensorrt_plan"
2 max_batch_size: 0
```

```
5 input [
6     {
7         name: "input"
8         data_type: TYPE_FP32
9         dims: [ 8, 16 ]
10    }
11]
```

```
13 output [
14     {
15         name: "output"
16         data_type: TYPE_FP32
17         dims: [ 8, 16 ]
18    }
19]
```

```
1 platform: "tensorrt_plan"
2 max_batch_size: 0
```

```
5 input [
6     {
7         name: "input"
8         data_type: TYPE_FP32
9         dims: [ -1, 16 ]
10    }
11]
```

```
13 output [
14     {
15         name: "output"
16         data_type: TYPE_FP32
17         dims: [ -1, 16 ]
18    }
19]
```


Inplace reshaping

```
1  platform: "tensorrt_plan"
2  max_batch_size: 8
3
4
5  input [
6      {
7          name: "input"
8          data_type: TYPE_FP32
9          dims: [ 1 ]
10         reshape: { shape: [ ] } # (batch_size, 1) -> (batch_size)
11     }
12 ]
```

Config Datatypes

Model Config	TensorRT	TensorFlow	ONNX Runtime	PyTorch	API	NumPy
TYPE_BOOL	kBOOL	DT_BOOL	BOOL	kBool	BOOL	bool
TYPE_UINT8	kUINT8	DT_UINT8	UINT8	kByte	UINT8	uint8
TYPE_UINT16		DT_UINT16	UINT16		UINT16	uint16
TYPE_UINT32		DT_UINT32	UINT32		UINT32	uint32
TYPE_UINT64		DT_UINT64	UINT64		UINT64	uint64
TYPE_INT8	kINT8	DT_INT8	INT8	kChar	INT8	int8
TYPE_INT16		DT_INT16	INT16	kShort	INT16	int16
TYPE_INT32	kINT32	DT_INT32	INT32	kInt	INT32	int32
TYPE_INT64		DT_INT64	INT64	kLong	INT64	int64
TYPE_FP16	kHALF	DT_HALF	FLOAT16		FP16	float16
TYPE_FP32	kFLOAT	DT_FLOAT	FLOAT	kFloat	FP32	float32
TYPE_FP64		DT_DOUBLE	DOUBLE	kDouble	FP64	float64
TYPE_STRING		DT_STRING	STRING		BYTES	dtype(object)
TYPE_BF16					BF16	

Version Policy

Доступны все версии модели

```
version_policy: { all { }}
```

Доступны только последние две

```
version_policy: { latest: { num_versions: 2 }}
```

Доступны первая и третья версии

```
version_policy: { specific: { versions: [1,3] }}
```

```
1  platform: "tensorrt_plan"
2  max_batch_size: 0
3
4
5  input [
6      {
7          name: "input"
8          data_type: TYPE_FP32
9          dims: [ -1, 16 ]
10     }
11 ]
12
13 output [
14     {
15         name: "output"
16         data_type: TYPE_FP32
17         dims: [ -1, 16 ]
18     }
19 ]
20
21 version_policy: { all { }}
```

Instance groups

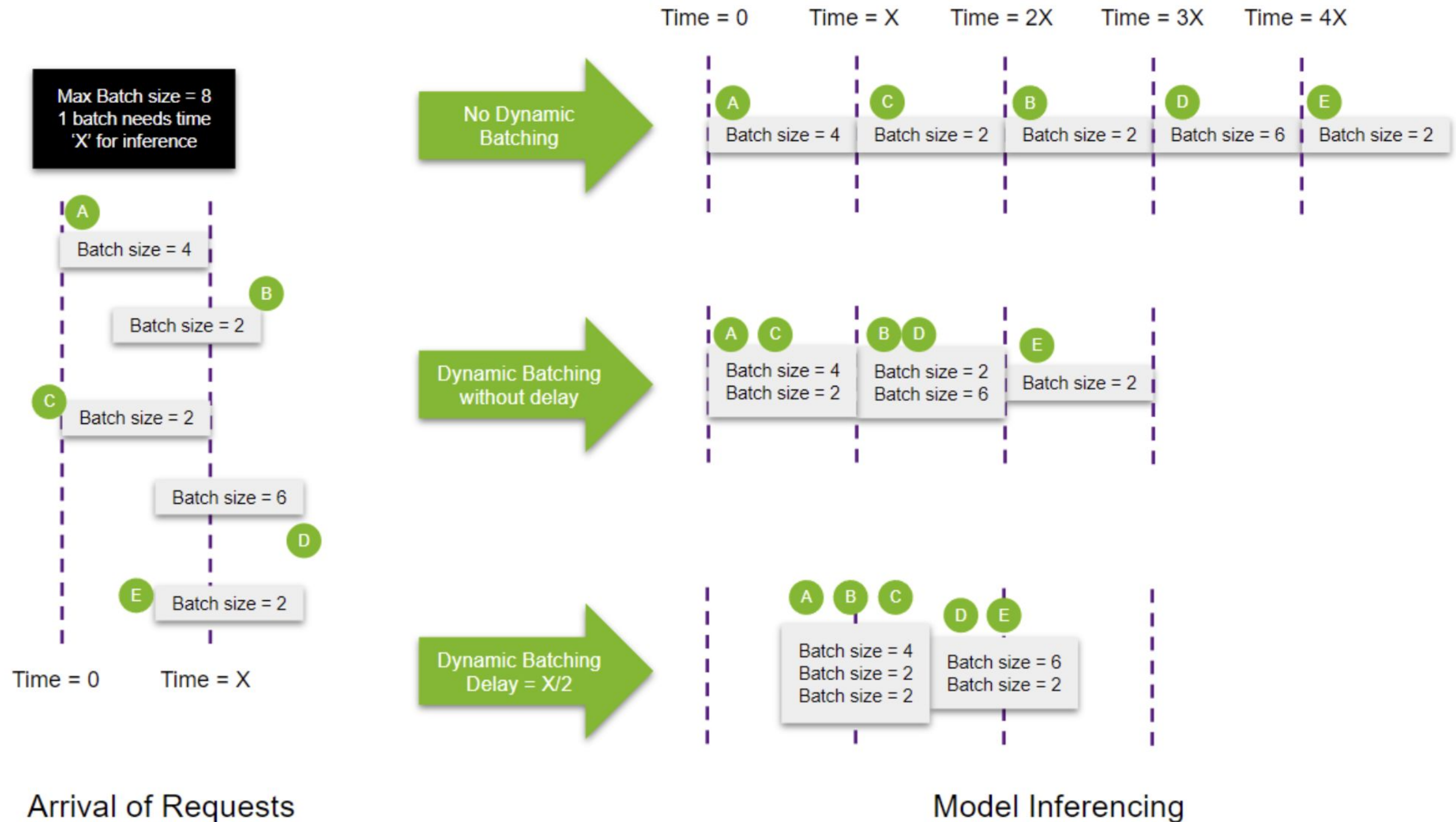
```
instance_group [  
  {  
    count: 2  
    kind: KIND_GPU  
  }  
]
```

```
instance_group [  
  {  
    count: 1  
    kind: KIND_GPU  
    gpus: [ 0 ]  
  },  
  {  
    count: 2  
    kind: KIND_GPU  
    gpus: [ 1, 2 ]  
  }  
]
```

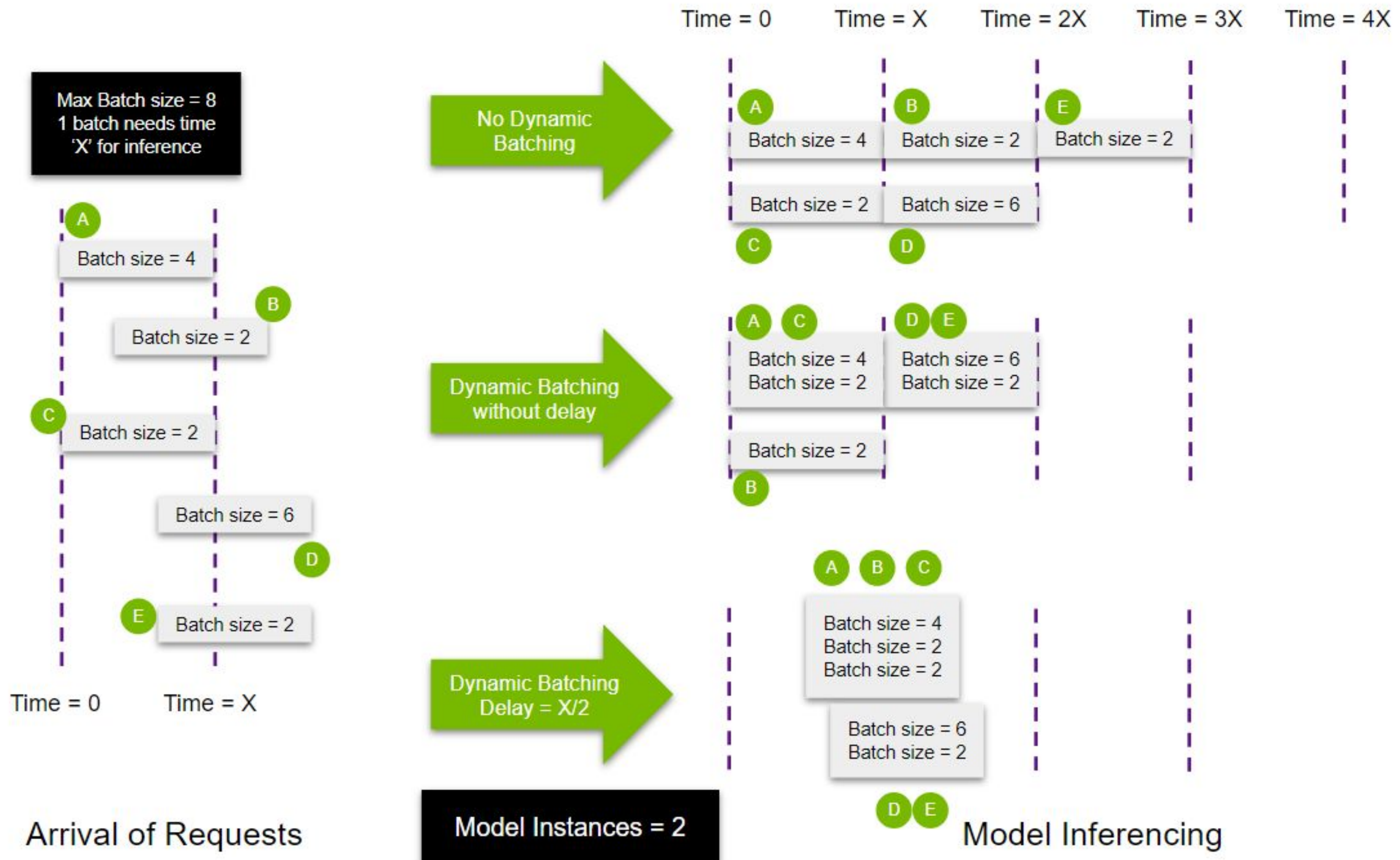

Dynamic batching

```
# Batching without delay  
dynamic_batching { }  
  
# Batching with max delay 0.1 ms  
dynamic_batching {  
|   max_queue_delay_microseconds: 100  
}  
  
# Batching without delay to closest size  
dynamic_batching {  
|   preferred_batch_size: [ 4, 8 ]  
}
```

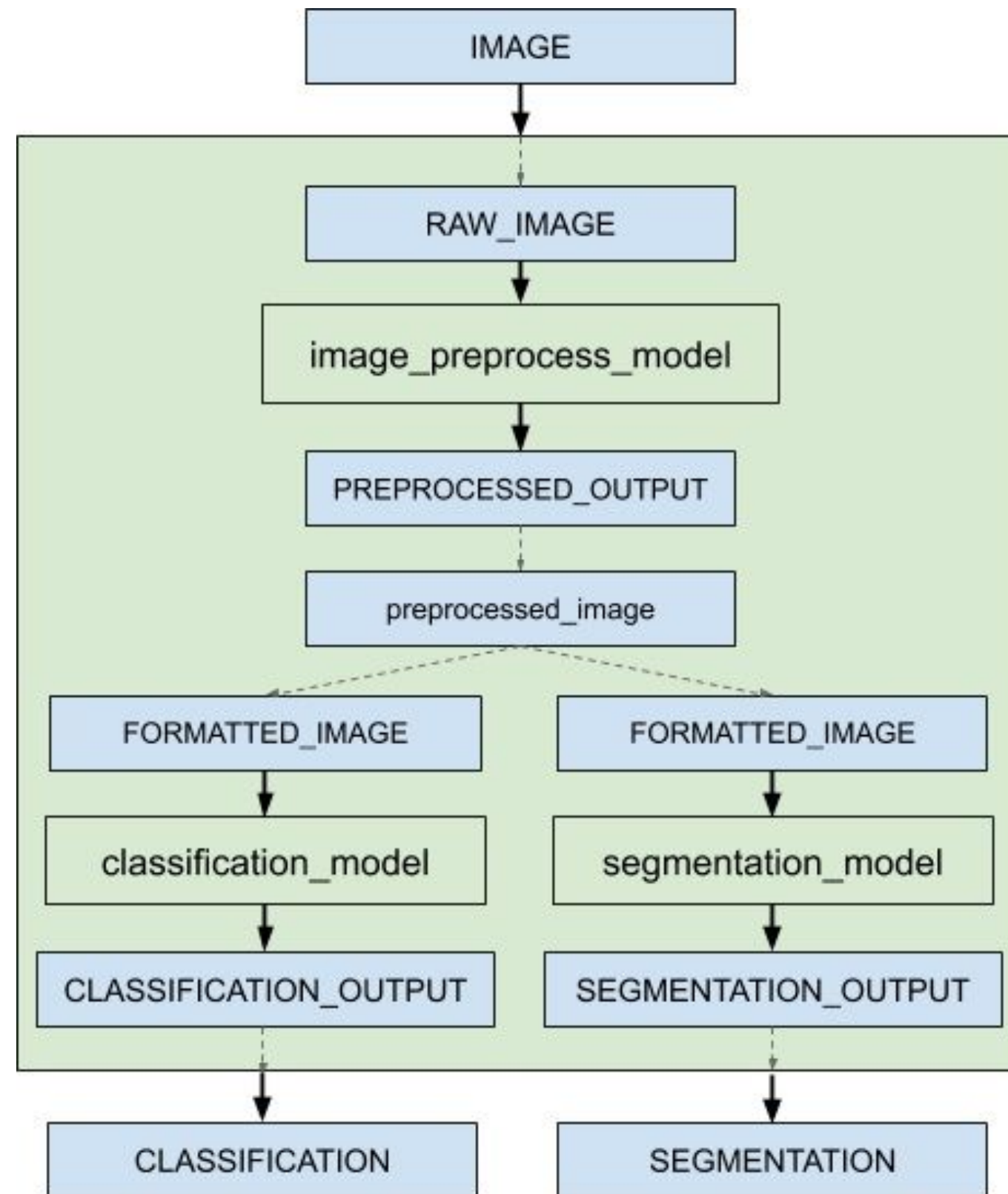

Dynamic Batching



Dynamic batching + Concurrent instance groups



Ensembling



Supported Backends

- TensorRT, TensorRT-LLM
- ONNX Runtime
- Tensorflow
- PyTorch
- OpenVINO
- Python
- Dali
- FIL (Forest Inference Library)
 - xgboost, lightgbm, sklearn, cuML
- create your own?

What else?

- Stateful and stateless models
- Ragged batching
- Decoupled mode
- Built-in rate limiter
- Response cache
- Model priority
-

Triton name ambiguity

1. Triton inference server
 - a. by Nvidia
2. Triton language
 - a. wrapper for CUDA API code
 - b. by OpenAI

These are unrelated projects!!!

UI for API *only* servvers

girafe
ai

03

UI constructors in Python

- Streamlint
 - Currently most popular solution
 - Needs page reload for many operations
 - tutorial [ru] <https://www.youtube.com/watch?v=C1ZPFwbleJ0>

Thanks for attention

Questions?

